Custom AI Health Agent

README.md

This project builds a custom AI health agent using **Groq's Llama3 model** and **CrewAI**. The agent fetches blood glucose levels from an external API and provides health insights and recommendations.

Project Overview

This AI health agent utilizes:

- **LangChain-Groq**: To interact with the Llama3 model for inference.

- **CrewAI**: To orchestrate the tasks and handle agent interactions.

- **RapidAPI**: To fetch real-time blood glucose data using the `human.p.rapidapi.com` API.

The agents are designed as:

- **Data Fetcher Agent**: Responsible for fetching blood glucose data.

- **Analysis Agent**: Analyzes the fetched data and provides recommendations.

Key Features

- **Real-time data retrieval**: Fetches blood glucose levels for a given date.

- **Health advice**: Provides personalized health insights and recommendations based on blood glucose levels.

- **Customizable agents**: You can modify the agents' roles and tasks for more specific use cases.

Installation

To use the AI Health Agent, you need to install the required dependencies. You can install them using the following commands:

```bash

pip install pip --progress-bar off

pip install 'crewai[tools]'==0.28.8 --progress-bar off

pip install langchain-groq==0.1.3 --progress-bar off
```

Setup

API Keys:

You need to get your **Groq API Key** and **RapidAPI Key**. You may store the keys in your environment variables using:

```bash

export GROQ_API_KEY="your_groq_api_key"

export RAPIDAPI_KEY="your_rapidapi_key"
```

Alternatively, you may clone the repo and replace the keys in the code.

API Call

The agent fetches blood glucose data from `human.p.rapidapi.com`. The API accepts a date parameter to return the corresponding glucose level.

Usage

Once you have the setup ready, you can run the AI health agent. The agent will:

- Retrieve blood glucose data for a specified date.
- Analyze the data and provide health advice based on the glucose level.

Example Query

You can input queries like:

- "What is my blood glucose level today?"

Customization

You can modify the roles and backstories of the agents or adjust the tasks for more personalized use cases. For instance, you can add tasks to provide additional health insights or track more health metrics.

Read The Full Blog

For a detailed explanation of the code, visit [How to Build a Custom AI Agent – ProjectPro](https://www.projectpro.io/article/how-to-build-a-custom-ai-agent/1096)

Custom_Medical_AI_Agent.ipynb

Notebook source cells (code preserved as provided; not executed)

Cell 1 — code

```
!pip -qqq install pip --progress-bar off
!pip -qqq install 'crewai[tools]'==0.28.8 --progress-bar off
!pip -qqq install langchain-groq==0.1.3 --progress-bar off
```

Cell 2 — code

```
import os
import requests
from langchain_groq import ChatGroq
from crewai import Agent, Task, Crew
```

Cell 3 — code

```
# Set API keys (Replace with your actual keys)
os.environ["GROQ_API_KEY"] = "GROQ API Key"
os.environ["RAPIDAPI_KEY"] = "RAPID API Key"
```

Cell 4 — code

```
# Llama 3 with Groq for inference
llm = ChatGroq(temperature=0, model_name="llama3-70b-8192")
```

Cell 5 — code

```
# Function to Fetch Blood Glucose Data from API and Return as a String
def get_blood_glucose(date):
    url = f"https://human.p.rapidapi.com/human/blood_gluecose/{date}"
    headers = {
        "x-rapidapi-host": "human.p.rapidapi.com",
        "x-rapidapi-key": os.getenv("RAPID API Key")
    }
    response = requests.get(url, headers=headers)

    if response.status_code == 200:
        data = response.json()
        glucose_level = data.get("glucose_level", "No data available")  # Adjust based on response structure
        return f"Blood glucose level on {date}: {glucose_level}"
    else:
        return f"Error: {response.status_code} - {response.text}"
```

Cell 6 — code

```
# Define the Data Retrieval Agent
data_agent = Agent(
    role="Health Data Fetcher",
    goal="Retrieve real-time blood glucose data from the API.",
    backstory="An AI health assistant that monitors blood sugar levels.",
    allow_delegation=False,
    verbose=True,
    llm=llm,
    memory=True,
    max_iter=3  # Avoids Infinite Loops
)
```

Cell 7 — code

```
# Define the Analysis Agent
analysis_agent = Agent(
    role="Health Advisor",
    goal="Analyze blood glucose levels and provide recommendations.",
    backstory="A virtual AI doctor specialized in glucose monitoring and diabetes care.",
    verbose=True,
    llm=llm
)
```

Cell 8 — code

```
# Define Blood Glucose Data Retrieval Task
fetch_data_task = Task(
    description="Fetch blood glucose level for a given date.",
    agent=data_agent,
    execute_fn=lambda: get_blood_glucose("2025-02-06"),  # Example date
    expected_output="string"  # Specify expected output type as string
```

```
)

# Define Analysis Task
analyze_task = Task(
    description="Analyze the blood glucose levels and give a recommendation.",
    agent=analysis_agent,
    expected_output="string"  # Ensure expected output is defined for analysis task
)
```

Cell 9 — code

```
# Create Crew with Agents
crew = Crew(
    agents=[data_agent, analysis_agent],
    tasks=[fetch_data_task, analyze_task],
    verbose=True
)

# Run the AI Health Agent with the correct format for inputs
query = {"query": "What is my blood glucose level today?"}
response = crew.kickoff(query)  # Pass query as a dictionary
print("\nFinal AI Response:", response)
```

Cell 10 — code

(empty cell)